上 海 电 力 大 学

工业控制网络技术与应用课程报告

项目名称：基于 ROS 2 与 ResNet18 的智能驾驶小车感知与控制设计

姓　名：_____

2026 年　11 月　6 日

评语：	成绩：

目录

1 绪论.....2
1.1 研究背景.....2
1.2 课题任务与目标.....3
1.3 主要工作与技术路线.....3
2 系统总体方案.....4
2.1 硬件平台与软件环境.....4
2.2 软件架构与话题数据流.....5
2.3 接口设计.....6
3 车道线识别模型的数据、训练与部署.....7
3.1 数据采集与标注工具.....7
3.2 数据集构成与标签分布.....8
3.3 训练方法与过程.....9
3.4 导出、转换与车端部署.....11
3.5 坐标一致性：从 224×224 回到像素.....13
4 交通标志识别与速度策略.....14
4.1 从静态图练习到车上节点.....14
4.2 方法选择：为什么不用小模型.....15
4.3 颜色空间与阈值选择.....16
4.4 形态学处理与面积判据.....17
4.5 帧间防抖与信号保持.....17
4.6 控制端的速度覆盖策略.....18
5 激光雷达避障与 IMU 调头.....19
5.1 前向扇区扫描与点簇判据.....19
5.2 偏航角积分与 PD 闭环.....19
5.3 四状态机与失效保护.....20
6 系统联调与问题分析.....21
6.1 部署流程与自检.....21
6.2 PID 调参的方法论.....21
6.3 典型问题与处理.....25
6.4 与课题目标的对应关系.....26
7 结论与展望.....27
7.1 结论.....27
7.2 不足与展望.....27
8 参考文献.....28

1 绪论

1.1 研究背景

智能驾驶是人工智能在物理世界中最具代表性的落地方向之一。它之所以难，不在于某一个算法，而在于感知、决策与控制必须在真实的时延、噪声与算力约束下同时成立：摄像头给出的是一帧带噪的像素，雷达给出的是一圈稀疏的距离点，底盘能执行的只有左右轮的转速差，而这一切都要在几十毫秒内走完一次闭环。任何一环的偏差，最终都会以“车压线”或“车停在半路”的形式暴露出来。

过去十余年，智能驾驶的技术路线经历了从“规则驱动”到“数据驱动”的迁移：感知端由人工设计的特征转向深度网络，决策端由有限状态机转向学习与优化，执行端则由分散的控制器走向域集中。但对教学与研究而言，直接在量产车上验证想法既不现实也不经济；更常见的做法是搭一个小尺度平台，把完整链路压缩到一台小车、一块嵌入式算力板和一个受控场地上，让“感知—决策—控制”的每一次耦合都能被观察、被复现。

本课题沿着这条思路展开：以一台 Originbot 差速底盘小车为平台，主控为地平线 RDK X5，运行 Ubuntu 22.04 与 ROS 2 Humble，在其上实现车道线识别、交通标志响应、障碍识别与调头三类典型能力，并把它们接进同一条控制回路。与“只调通一个模型”不同，本课题更关心三个问题：模型怎样从数据长出来、它在车端的时延与精度如何权衡、以及某一环失效时系统如何安全退出。

本报告以工作区中的 ros2_last 工程为对象，说明这台小车是怎样从“相机拍到线”走到“车沿线路面行驶”的：包括数据与标注的约定、模型的训练与部署、颜色分割与激光雷达的参数选择，以及控制层的状态机与失效保护。报告中的参数、话题名与阈值均取自工程源码，统计量由工作区数据集直接计算得到。

1.2 课题任务与目标

课题把“让小车在受控场地上自主跑完一圈”这一总目标拆成四项可以独立验证的任务：车道循迹、交通标志响应、障碍识别与调头、运行效率。四者并非并列关系——循迹是基础，标志与调头是叠加在循迹之上的动作，效率则是在前两者稳定之后才有意义的优化目标。表 1 给出了每项任务的具体要求、工程实现与验证方式。

表 1　课题任务目标与本工程的对应设计

任务目标	具体要求	本工程的实现与验证方式
车道循迹	沿车道中心线连续行驶，正向与逆向各完成一圈，全程不脱离车道	ResNet18 线心回归 + 30 Hz 循迹 PD + 四状态机；以跑圈录像与线心偏差记录验证
交通标志响应	识别禁行、儿童、停车三类标志，分别执行减速与停车	HSV 颜色分割 + 面积判据 + 帧间防抖 + 速度覆盖；以静止与行驶两种工况的触发记录验证
障碍识别与调头	检出前方近距障碍，完成 180° 调头并沿原路返回	激光雷达扇区聚类 + IMU 偏航闭环；以不同距离桩桶的调头成功率验证
运行效率	在不脱离车道的前提下缩短单圈用时	速度-转角耦合、变化率限幅与返程降速；以单圈耗时统计验证

这四项目标之间存在明确的张力。循迹要求“跟得紧”，效率要求“跑得快”，而跟得越紧、跑得越快，图像模糊与线心跳变就越严重；标志响应要求“反应果断”，但阈值放得太松又会把地面反光当成标志，把车钉在原地。工程上的大部分取舍——增益取多大、限幅卡在哪、标志保持多久——本质上都是在这几对矛盾之间找平衡点，第 6 章会逐项展开。

1.3 主要工作与技术路线

围绕上述四项目标，本工程把任务拆成四条相对独立的链路，分别对应一个或一组 ROS 2 节点。

（1）循迹感知：用 ResNet18 直接回归车道线中心点的像素坐标，替代传统的阈值分割加霍夫变换；后处理不做任何几何拟合，网络输出即为控制量。

（2）标志识别：在 HSV 空间做三路颜色分割，配合形态学去噪、轮廓面积判据与帧间防抖，输出 0/1/2/3 四类信号。

（3）避障与调头：激光雷达前向扇区聚类判定障碍，检出桩桶后由 IMU 偏航角闭环完成 180° 原地调头。

（4）运动控制：30 Hz 定时器驱动的循迹 PD 控制器、速度-转角耦合与变化率限幅，统一由四状态机管理，并对每一路输入设置超时判据。

这条技术路线可以概括为“用回归代替检测、用几何代替学习”。车道线是连续、弱纹理的细长目标，先用检测框框住再做拟合，误差会经过两级放大；把线中心直接当作回归目标，网络只需回答“线在哪”，后处理没有中间环节，实时性更好。反过来，交通标志颜色鲜明、形状固定，用深度学习反而增加部署成本与不确定性，几何方法更合适。

2 系统总体方案

2.1 硬件平台与软件环境

小车采用 Originbot 差速底盘，主控为地平线 RDK X5，运行 Ubuntu 22.04 与 ROS 2 Humble。感知外设包括一路 USB 摄像头（分辨率 1920×1080，默认设备 /dev/video8）、一台二维激光雷达和板载 IMU。RDK X5 片上集成 BPU 加速器，模型经地平线工具链编译后，由 dnn_node 框架调度到 BPU 上推理，CPU 只负责预处理与后处理。

相机链路做了共享内存优化：hobot_usb_cam 采集 /image，hobot_codec 将其解码为 NV12 后放入共享内存，以 /hbmem_img（HbmMsg1080P）发布，dnn_node 直接以此为输入。这样避免了每帧 1920×1080 彩色图像的 CPU 拷贝，对 30 Hz 的控制节拍来说是必要的；代价是整条链路被绑定在 NV12 格

式与共享内存机制上，换相机或换驱动时需要重新确认接口是否匹配。

2.2 软件架构与话题数据流

工程由三个 ROS 2 功能包组成。line_follower_perception 是 C++ 包，封装 dnn_node 完成模型加载、推理调度与后处理；originbot03_autonomy 是 Python 包，包含循迹控制、避障与标志识别三个节点；originbot03_bringup 只承载 launch 文件与参数配置，把四个节点组合成一次可复现的启动。

节点之间只通过话题耦合，数据流如图 1 所示。三个感知节点彼此独立，任一节点异常退出都不会阻塞其余节点；控制节点对每一路输入都设置了超时判据，超时即停车，而不是沿用旧值继续行驶。这种“松耦合 + 输入端兜底”的结构，使得单个模块可以在不触碰其他模块的前提下替换——例如把 HSV 标志识别换成 YOLO 检测，只需保持 /traffic_sign 的语义不变。

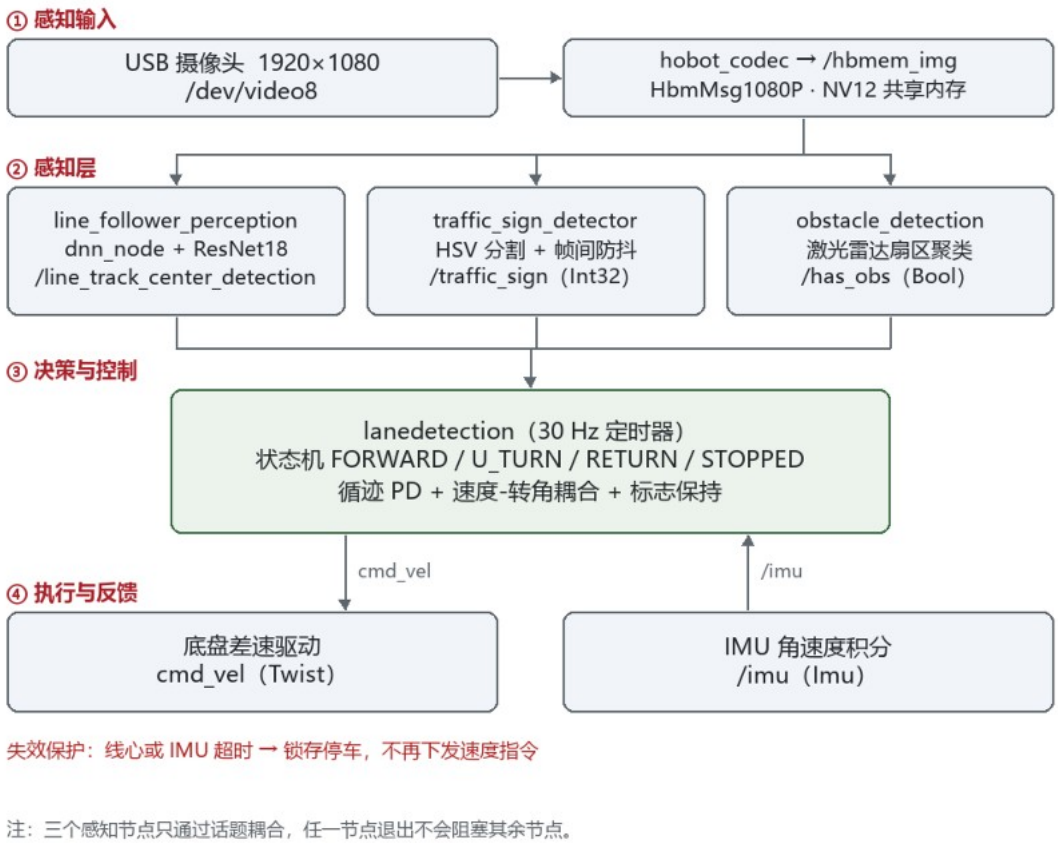


图 1　系统总体架构与 ROS 2 话题数据流

启动层面，perception.launch.py 一次性拉起四个节点，并把模型路径作为 launch 参数暴露出来，默认指向 share/line_follower_perception/models 下的 .bin；需要替换模型时不必改代码，只需在命令行覆盖 model_path。camera_pipeline.launch.py 则用于接管相机：它包含 hobot_usb_cam、hobot_codec 与 hobot_shm 三个子 launch，把 USB 相机输出解码为共享内存 NV12，供只看 /hbmem_img 的感知节点使用。

2.3 接口设计

表 2 列出了四个节点的全部输入输出。接口数量不多，但有三处约定必须严格对齐，否则会出现“节点都在跑、车却不动”的现象。

表 2　ROS 2 节点与话题接口

节点	订阅	发布
line_follower_perception	/hbmem_img（HbmMsg1080P）	/line_track_center_detection（PointStamped）
obstacle_detection	/scan（LaserScan）	/has_obs（Bool）
traffic_sign_detector	/image（CompressedImage）	/traffic_sign（Int32）
lanedetection	线心、/has_obs、/imu、/traffic_sign	/cmd_vel（Twist）

第一处是消息类型。本工程的标志识别节点订阅 CompressedImage，而部分相机驱动直接发布 sensor_msgs/Image；两者不能混用，需要插入压缩或重发布节点，而不是在代码里静默改类型，否则最终只会表现为“回调从不触发”。

第二处是坐标系。线心话题的 header.frame_id 固定为 image_pixels，x、y 都是原图像素；控制节点以 960 作为图像中心计算偏差。一旦某一端改成归一化坐标或 ROI 内坐标，控制律的增益含义就变了，表现为“越修越偏”。

第三处是语义。int32 的 /traffic_sign 用 0/1/2/3 区分正常、减速、蓝牌停车、红牌停车，其中 2 与 3 都表示停车但生效条件不同（2 只在返程生效）。这类“魔法数字”必须在文档里写清楚，否则后续维护极易误解。

3 车道线识别模型的数据、训练与部署

3.1 数据采集与标注工具

车道线的数据集不是现成的，而是自己录、自己标出来的。小车在场地上由人遥控走一遍，机载相机把画面录成视频；随后用标注脚本 originbot_data.py 把视频逐帧回放，由人在画面上点出车道线的中心位置。整个采集过程不依赖任何外部标注平台。

标注界面做了两件事来降低出错率。一是把 ROI（第 500～948 行）以外的区域压暗到 30%：脚本先用 addWeighted 与掩码把整帧调暗，再把 ROI 原样贴回去，于是标注者的注意力被限制在真正参与训练的那块画面里。二是点击位置做了行范围判断，落在 ROI 之外的点击直接忽略，从源头上杜绝了越界坐标。

一次有效的左键点击会做四件事：以点击点为圆心画一个半径 25 像素的红点作为即时反馈；按 xy_{x}_{y}_{uuid} 的格式生成文件名；把该帧的 ROI 裁块（1920×448）以这个名字存进 image_dataset；暂停 0.5 s 再继续播放，避免手抖连点。四个动作都在同一个回调里完成，标注与存储不会脱节。

标完还要复核。annotate_image_dataset.py 遍历 image_dataset，从文件名解析出 (x, y)，把 y 减去 ROI 起始行 500 换算到裁块坐标，再在图上画半径 10 的点与坐标文字，输出到 image_dataset_marked。这份复核图不参与训练，唯一用途是让人一眼看出“点有没有点在线上”。

这套工具最大的好处是标签与图像同源：图像是裁块，标签是裁剪前的绝对坐标，两者由同一段代码产生，不存在“先裁剪、再按裁块坐标标注”导致的系统性偏移。此外，文件名即标签，训练脚本不需要解析任何标注文件，用 glob 加一条正则就能把图像与标签配对。



图 2　车道线数据集的原始 ROI 裁剪图与标注复核图

3.2 数据集构成与标签分布

工作区的 data/image_dataset 中共有 752 张图像，均为 1920×1080 的原始帧裁去上半部分后的结果。车头前方约 3～5 m 的路面才包含有效的车道线信息，因此只保留第 500～948 行作为感兴趣区域（ROI），裁剪后为 1920×448。另一份 data/image_dataset_marked 共 169 张，是在同一批图像上叠加红色标记点的复核图，只用于人工检查标注质量，不参与训练。

对 752 个标签做统计（图 3）：横坐标 x 落在 1～1910 之间，均值 1035、标准差 654；纵坐标 y 落在 565～947 之间，均值 814、标准差 106。可以看到两个特点：一是 x 的分布并不对称，落在左半幅（x < 960）的样本仅占 34.6%，这与场地以左转弯为主、采集时车辆多贴着右侧线行驶有关；二是 y 分布集中在 800 附近，因为线中心多数出现在图像下部。

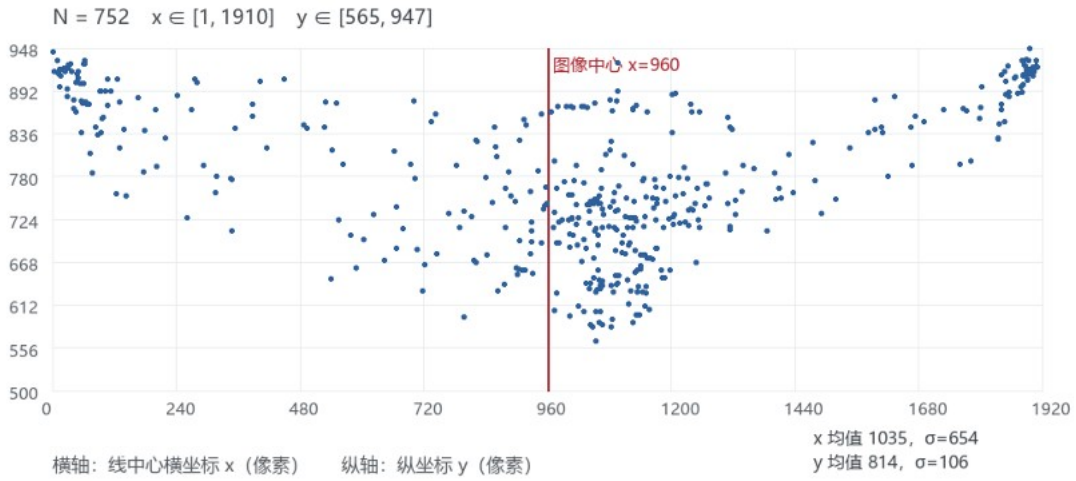


图 3　752 个标注样本在 ROI 内的标签分布

针对样本偏斜，训练脚本在数据增强中做了水平翻转，并把标签的 x 取反。在归一化坐标系下，这等价于左右镜像，可以有效缓解“左侧样本少”的问题；再配合强度 0.3 的色彩抖动模拟光照变化，使得在只有 752 张图的情况下，单个 epoch 能看到的数据分布接近翻倍。

这种处理也有代价：翻转会引入“镜像世界”的样本，如果场地左右并不对称（例如标志牌只出现在单侧），模型可能学到并不存在的对称性。因此增强只作用于训练集，验证集保持原始分布；同时翻转概率取 0.5 而不是 1.0，保留一半原始朝向的样本。

3.3 训练方法与过程

训练在 Windows 工作站上完成，软件栈是 PyTorch + torchvision，显卡为 NVIDIA GeForce RTX 5070 Laptop（计算能力 sm_120）。这里踩过两个环境坑。第一个是 PyTorch 装成了 CPU 版，torch.cuda.is_available() 返回 False，训练一开始只能退回 CPU，200 个 epoch 根本跑不完；换成与 CUDA 12.8 匹配的轮子后 GPU 才真正参与计算。第二个是 DataLoader：图像被预载进显存之后，CUDA 张量不能跨进程共享，num_workers 必须设成 0，否则子进程会直接报错退出。

数据加载没有走常规的 Dataset + 多进程读取，而是“一次性预载”：脚

本先把 752 张图统一定尺到 224×224、转张量、按 ImageNet 的均值方差归一化，然后整块搬进显存（preload_to_gpu）。之后每个 epoch 只做 clone 与增强，不再碰磁盘。代价是显存占用，好处是把数据管线从训练循环里彻底摘出去——否则 batch 只有 24，每个迭代步都在等 CPU 解码。

增强全部在 GPU 张量上完成：以 0.5 的概率水平翻转（标签 x 同步取反），再做亮度、对比度、饱和度各 ±30% 的随机抖动，色调抖动幅度 0.3。水平翻转是针对 3.2 节“左半幅只占 34.6%”的补丁；色彩抖动则是对场地白线在不同时段偏冷偏暖的模拟。因为增强只做张量运算，不涉及图像编解码，这部分几乎不占时间。

网络用 ResNet18，加载 ImageNet 预训练权重，把最后的全连接层由 1000 类改为 2 个输出，直接回归归一化后的线心坐标 (x, y)。损失函数是均方误差，优化器是 Adam（默认学习率）。预训练权重的作用是让骨干网络一开始就具备边缘与纹理的通用特征，避免在 752 张图上从零学起。

训练循环本身不复杂：批大小 24，最多 200 个 epoch；每个 epoch 先在训练集上前向、反传、更新参数，再在验证集上前向计算一次损失；只有当验证损失低于历史最优时才保存权重（best_line_follower_model_xy.pth）。日志一行一个 epoch，形如 Epoch:0,Train loss:0.0xx,Val loss:0.0xx，据此判断是否收敛、是否过拟合。

用验证损失而不是训练损失来选模型，是为了避免“训练集背下来了、场地上却过拟合”。训练集与验证集按 9:1 随机划分，划分在预载之后用 random_split 完成，因此每张图只会出现在一侧。

需要说明的是，工程内还保留了 tools/train_line_model.py 作为精简复现版：50 个 epoch、CPU 侧增强、固定随机种子 42。两者流程一致，只是轮数与数据管线不同；日常小改数据时用精简版更快，正式重训则用上面这份“实战版”。

表 3　ResNet18 线心回归模型的训练配置

项目	取值	说明
骨干网络	ResNet18	加载 ImageNet 预训练权重
输出	2 维（归一化 x, y）	全连接层由 1000 类改为 2 维
输入	3×224×224	ImageNet 均值方差归一化
损失函数	MSE	对两个方向等权
优化器	Adam	使用默认学习率
批大小 / 轮数	24 / 200	精简版为 50 轮
数据管线	全量预载入显存	num_workers = 0
数据划分	9 : 1 随机划分	random_split，验证集不增强
数据增强	水平翻转 + 色彩抖动	翻转概率 0.5，标签 x 取反
模型选择	按验证损失保存最优	best_line_follower_model_xy.pth

训练结束后还有一步定性验证：originbot_validate.py 把 .pth 加载进来，在录制的场地上逐帧推理，画面上会同时显示预测点、ROI 边框、实时帧率，并另开一个 ROI 特写窗放大预测点与线的相对位置，按空格可以暂停逐帧看。这套工具不能给出像素误差，但能快速暴露“点系统性偏左”“转弯时点跳变”这类结构性问题。若要给出定量指标，应当从标注集里留出一份带真值的测试集，统计像素平均绝对误差，这一步在本工程中尚未做。

3.4 导出、转换与车端部署

训练产出的是 PyTorch 权重（.pth），部署前要先导出 ONNX，再由与板端镜像配套的地平线工具链编译成 .bin。这一步是整个流程里最容易卡住的地方，工程目录里为此留下了六个脚本，恰好记录了踩坑的全过程。

第一次用 torch.onnx.export 指定 opset 11 导出，但 PyTorch 版本较新，导出的模型 ir_version 实际是 10、部分算子仍按 opset 18 的语义写出。量化工具链不认：它只接受较老的 IR 与 opset，遇到新属性直接报错。

于是有了 downgrade_onnx_opset.py：它把 opset 强制改回 11、ir_version 改成 9 后另存，得到一个“名义上合规”的模型。但只改版本号并不够，算子

结构仍是新版本的：fix_onnx_compat.py 逐个修正——ReduceMean 在 opset 18 里允许把 axes 作为第二个输入，opset 11 里必须是属性，因此要删掉这个输入；Reshape 的 allowzero 属性与 ReduceMean 的 noop_with_empty_axes 属性在 opset 11 中不存在，要删掉；少一个输入的 Reshape 还要补上 shape 常量。此外 merge_onnx.py 负责把 .onnx 与 .onnx.data 两个文件合成单文件权重，因为工具链只吃单文件。

这六个脚本的存在本身就是结论：模型转换不是一条命令，而是“导出—检查 ir/opset—降级—删新属性—再校验”的反复迭代。每一步都用 onnx.checker.check_model 兜底，避免把一个“看起来能打开”的模型交给量化器，最后才发现是算子不兼容。整条流水线如图 4 所示。



图 4　从数据采集到车端部署的模型流水线

车端节点的处理链路是：订阅 /hbmem_img，先校验分辨率与数据长度（要求不小于 1920×948）；再用 hobot_cv 在 NV12 域直接裁剪第 500～948 行，缩放到 224×224 并生成 pyramid 输入；随后调用 Run() 提交推理，在回调里读取输出张量的前两个浮点数。整个过程中彩色转换被完全跳过，模型看到的始终是 NV12 数据。

这里有一个必须与训练侧对齐的细节：NV12 的宽度按 16 字节对齐，张

量的 alignedShape 与 validShape 可能不同，按对齐后的行宽去读像素而不按 stride 计算偏移，图像会整体错位。本工程裁剪后固定为 224×224，而 224 恰好是 16 的整数倍，从根上绕开了这个问题；换成 222 或 230 之类的尺寸，就必须显式处理行对齐。

3.5 坐标一致性：从 224×224 回到像素

模型输出是归一化值，必须反变换回像素才能给控制器使用。横坐标的逆变换是 x = (x + 1) × 960，纵坐标的逆变换是 y = 500 + (y + 1) × 224，其中 x、y 为网络输出的归一化值，500 是 ROI 的起始行。

初版示例节点在这里犯了两处错：它裁剪的是第 80～304 行——那是画面偏上的部分，而训练数据来自第 500～948 行；反变换写的是 80 + (y + 1) × 112，把 224 行的 ROI 当成了半高 112。也就是说，模型在车上看的是和训练时完全不同的画面，输出的 y 还按一半的高度折算。这类错误不会报错、不会崩溃，只会表现为“点总是飘在天上、车永远在纠正一个不存在的偏差”。本工程把裁剪区间改成 [500, 948]、反变换改成 500 + (y + 1) × 224，与训练脚本里的 ROI_TOP、ROI_BOTTOM 一一对应，并在代码里用同一组常量表达，避免两处各写一套。

修正后的解析器同时做了三件事：按上式反变换；检查输出是否为有限值；检查归一化输出是否落在 [−1, 1] 之内。任一条不满足就丢弃该帧，而不是把异常值送进控制环。这一条看似琐碎，实际上避免了一次“模型偶发 NaN 导致小车猛打方向”的隐患——在高速行驶时，单帧异常足以让车离开场地。

最后需要如实说明验证边界。工程中随附的 .bin 是课件提供的成品模型，其训练时的裁剪方式与归一化约定无法从二进制里读出。本工程已经把训练脚本与 C++ 侧的 ROI、归一化方式统一到同一套约定，但成品 .bin 是否由这套约定编译而来，无法证明。因此在实车调试前，应当先用同一段视频比对“PC 端 PyTorch 推理”与“车端 .bin 推理”的输出；若偏差明显，应重新训练并编译，而不是直接上路。

4 交通标志识别与速度策略

4.1 从静态图练习到车上节点

标志识别这条链路是从课堂练习长出来的。练习给了一张 1280×720 的红绿灯照片和一段待补齐的代码：截取 img[0:230, 600:1280] 作为观察区，对三路颜色分别做 inRange 得到掩膜，再用 findContours 取外轮廓、contourArea 算面积，比一比哪一路的最大轮廓最大，就在原图上画一个 boundingRect 并打印判定结果。图 5 用同一组阈值在 redlight.jpg 上复现了这个过程。

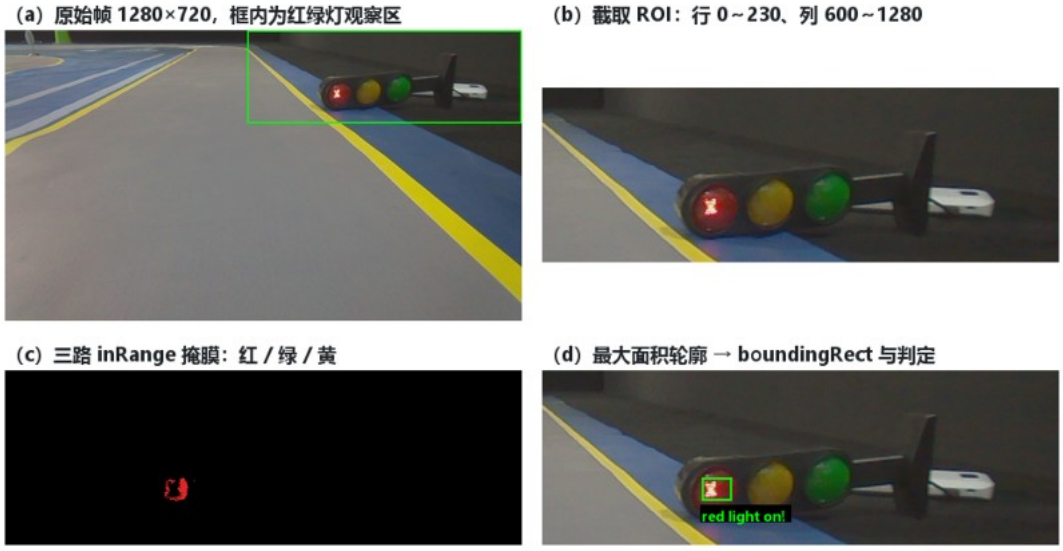


图 5　课堂红绿灯识别流程的复现：ROI、颜色掩膜与轮廓判定

复现结果很直观：三路掩膜里只有红色那一路留下了光斑（390 个像素），绿、黄两路是空的，于是程序判定“red light on!”并框出红灯。这套“谁的面积大谁亮”的规则虽然朴素，却已经包含了颜色分割类识别的基本骨架：限定区域、分离颜色、连通成块、比大小、给结论。

练习代码有两处值得记录。第一，medianBlur(7) 算出来了却没被后续使用——掩膜直接取自未模糊的原图 ROI，这类“写了但没接上”的疏漏在从练习走向工程的过程中很常见；第二，它用 BGR 三分量阈值（红色的 [0,0,120]～[50,80,255]）直接做 inRange，而不是先转 HSV。BGR 的三个分量都与亮度耦

合，换一块场地、换一个时段就得重新标定；这正是示例节点改成先 cvtColor 到 HSV 的原因。

把流程搬到车上的示例节点 traffic_sign.py 是这样的：订阅压缩图像，取画面最上面 50 行当作观察区（红绿灯通常在天上），转 HSV，红绿两路 inRange 后各做一次 medianBlur(7)，然后用 np.max(mask) == 255 判断“掩膜里有没有该颜色的像素”，有就发布 True。

这个判据在实车上不可用：只要有 1 个像素落在阈值内就会触发，地面反光、衣服颜色、车身贴纸都能让它误报。工程版因此把“有没有”换成“有多大、并且连续几帧都大”——面积阈值、形态学去噪与帧间防抖这三件事，就是为了把“单像素存在性”升级成“稳定目标”。

4.2 方法选择：为什么不用小模型

一个自然的疑问是：既然 ResNet18 已经在车端跑起来了，为什么标志识别不也换成一个小型目标检测模型（例如 YOLO 系列的 n 级模型）？毕竟“统一到一套深度模型”在工程上更整齐，也能省掉逐个标志调阈值的工作。

原因在于两类任务在时间尺度上的差别。车道线是高频量：控制节点每 33 ms 取一次线心，线心晚一拍，车就已经偏出去了，所以它值得占用 BPU、走一条从共享内存直达推理的短链路。标志则是低频事件：一圈下来不过遇到三两次，晚 100 ms 做出反应，车也不会因此离开车道。把加速器让给高频任务、把低频任务留给 CPU 上的几何方法，是这套硬件上更合理的分工。

但如果只是“低频”，用 CPU 跑一个小模型似乎也行。真正的问题在延迟的构成：小模型的端到端链路包括图像格式转换与缩放、推理、NMS 后处理、结果序列化与话题传输，其中后处理与格式转换都落在 CPU 上，还要与 30 Hz 的控制循环争抢同一颗核。在本套硬件上，这条链路的端到端延迟已经超出控制周期允许的范围，画面中同时出现多个候选框时还会进一步拉长——表现出来就是“车已经开过标志牌，减速指令才到”。

颜色分割的链路则短得多：BGR 转 HSV、三路 inRange、一次开闭运算、一轮轮廓遍历，全部在 CPU 上顺序完成，没有模型加载、没有后处理排队，也不占用 BPU。它的代价是需要针对场地光照调整阈值，鲁棒性不如学习模型；但在这套硬件与这类任务上，“延迟确定、失败模式可解释”比“偶尔很准、偶尔很慢”更有价值。

这也是整个工程的一条方法论：按时间尺度分配算力。需要逐帧更新且对延迟敏感的量（线心）走加速器；事件型、可容忍数百毫秒的量（标志）走 CPU 几何方法；两者通过话题解耦，任何一方换实现都不影响另一方。图 6 把这两条链路的环节与资源占用画在了一起。

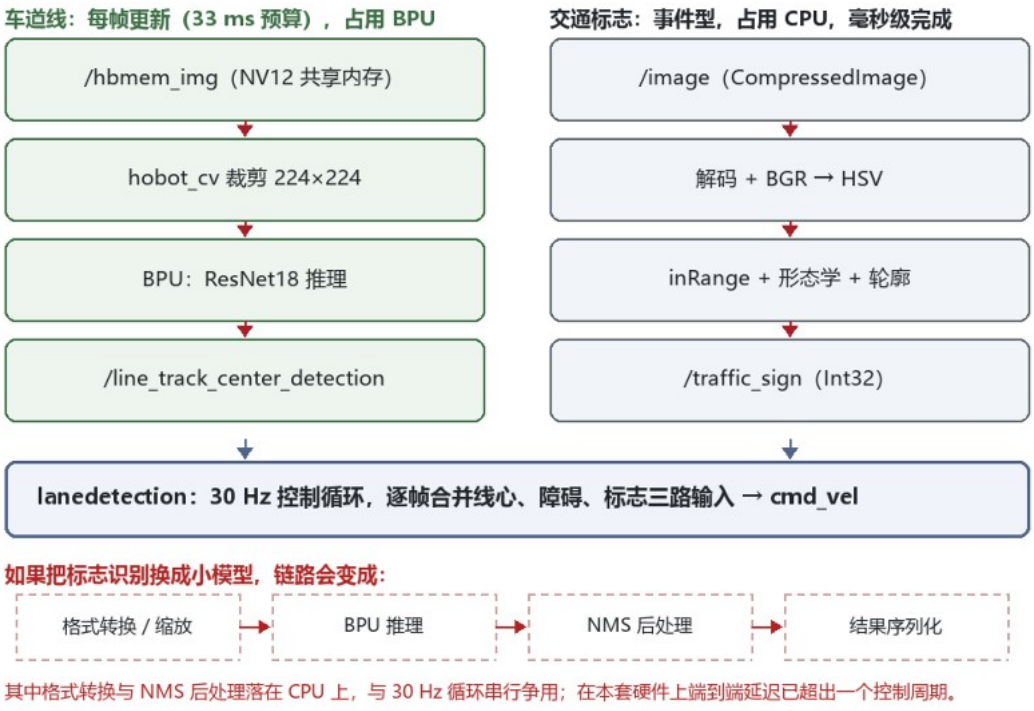


图 6　两类感知任务的处理链路与资源占用对比

4.3 颜色空间与阈值选择

场地上的三类标志（禁止通行、前方有儿童、P 停车）颜色差异明显、形状规则；如 4.2 节所述，深度模型在这套硬件上的时延并不划算，颜色分割反而是更稳妥的选择。节点选择在 HSV 空间做阈值分割：HSV 的 H 分量对光照强度不敏感，阴天与逆光下比 RGB 稳定得多。三组阈值见表 4。红色跨越

0° 与 180° 的分界，因此拆成两段区间再按位或。

表 4　交通标志的 HSV 阈值与面积判据

标志	H 范围	S 范围	V 范围	最小面积
黄（前方有儿童）	20 ～ 35	150 ～ 255	150 ～ 255	10
红（禁止通行）	0 ～ 5 与 170 ～ 180	150 ～ 255	150 ～ 255	600
蓝（P 停车）	100 ～ 130	100 ～ 255	50 ～ 255	600

阈值不是拍脑袋定的：S 与 V 的下限都提到 150，是为了滤掉路面反光与灰白色杂物；蓝牌的 S 下限降到 100，是因为打印的蓝色标牌饱和度普遍低于红色。这些数值需要在现场光照下再微调，但调整方向是明确的——宁可略紧，也不要放过地面杂色。

4.4 形态学处理与面积判据

阈值分割之后仍有噪点与孔洞，因此用 5×5 的核先做开运算（去掉孤立小白点），再做闭运算（填补标志内部的孔洞）。随后用 findContours 取外轮廓，用 contourArea 做面积判据：红、蓝的最小面积取 600 像素²，黄色放宽到 10 像素²。

黄色阈值之所以比红蓝低两个数量级，是因为两者的误判代价不对称。“前方有儿童”标志只要求明显减速，误触发一次减速损失的是几秒钟；而漏检导致直接冲过标志，这个目标就等于没完成。反过来，红蓝标志对应“停车”，误触发会把车钉在原地、损失更多，因此阈值取得更保守。这种按代价设置阈值的做法，在实车场景里比追求“准确率最高”更实用。

4.5 帧间防抖与信号保持

单帧判定在摄像机运动模糊、标志被遮挡时容易抖动，节点因此加了连续帧计数：同一类别连续 2 帧命中才触发。此外，黄色减速在触发后设置 1.5 s 的倒计时，倒计时内持续发布 1；蓝牌与红牌则保持最后一次检测值，直到出

现“无检测”帧才复位。

控制端还有第二层保持：lanedetection 收到非零标志后记录保持截止时间，在保持期内即使 /traffic_sign 变回 0，也继续按原标志执行。两层保持叠加，是因为从“摄像头看到标志”到“车头到达标志”存在几百毫秒的时延，没有保持机制就会出现“点一下刹车又立刻加速”的抽搐，既难看也容易被判为未减速。

4.6 控制端的速度覆盖策略

标志的执行落在控制节点的覆盖逻辑里：黄色把线速度限制到 0.3 m/s；蓝色与红色把线速度与角速度同时置零。这里有两个业务约束：其一，蓝色标志只在返程状态生效，因为只有返程才会经过 P 停车点；其二，返程起步的前 10 s 忽略所有标志，避免出发点附近的标志牌或地面反光把车“钉”在原地。

这两条约束本质上是场地知识而非算法，也说明在实际运行中，“什么时候不该检测”和“怎么检测”同样重要。完整的判定流程如图 7 所示，从订阅图像到覆盖速度指令，一共经过分割、去噪、轮廓、优先级、防抖、发布、保持七个环节，每个环节都只为下一环节提供一个更确定的输入。

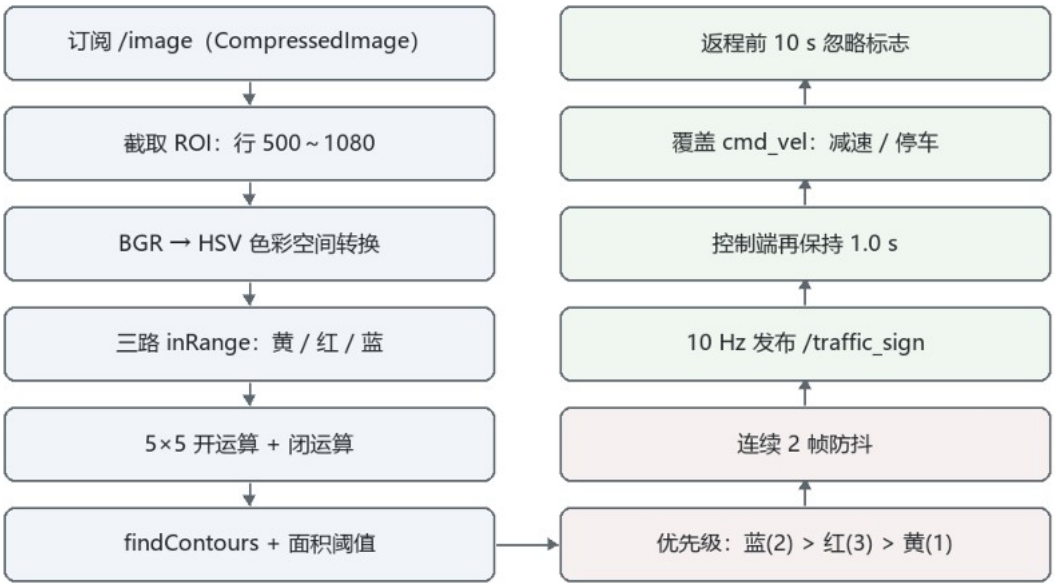


图 7　交通标志颜色分割与判定流程

5 激光雷达避障与 IMU 调头

5.1 前向扇区扫描与点簇判据

避障节点只关心车头前方 ±0.25 rad 的扇区。对扇区内的每一束激光，先做距离有效性判断（必须是有限值，且落在 0.1～0.4 m 之间），再按索引是否连续聚成点簇；只有连续 3 个及以上点同时满足近距离条件，才判定为障碍并发布 /has_obs 为真。

用“连续点数”而不是“总点数”作为判据，是为了区分一面墙和几根草。单点噪声、地面反光、细杆的回波在索引上通常不连续，用簇长度可以自然滤掉；而桩桶、纸箱这类实体目标会在角度上形成连续的一段，很容易超过 3 个点。0.4 m 的距离阈值对应“车身半宽加安全余量”，0.25 rad 则对应车头宽度在 3 m 外的张角。

节点还做了两处防御：若收到的 LaserScan 为空或角度增量为 0，直接发布“有障碍”；若扇区内没有任何点，同样发布“有障碍”。这两处的默认值都偏向安全侧——宁可误停，也不要在传感器异常时继续前进。

5.2 偏航角积分与 PD 闭环

检测到桩桶后，控制节点从往程切到调头状态，先清零累计偏航角与线速度，然后对 IMU 的 z 轴角速度做时间积分，得到已经转过的角度。控制律是以 π 为目标、以累计偏航角为反馈的 PD 控制，比例增益 2.0、微分增益 0.5，输出限幅在 0.2～0.6 rad/s，到位容差 0.06 rad（约 3.4°）。

三个工程细节值得说明。第一，角速度下限设为 0.2 rad/s 而不是 0，否则在接近目标时电机会“爬行”，轮胎与地面的静摩擦会让车停在半路，而此时累计角度尚未到位，逻辑上会一直卡住。第二，旋转方向固定为正，不随 PD 输出的符号翻转，避免在目标附近来回摆动。第三，调头全程线速度为 0，即原地旋转，这在 2 m 左右宽的场地上比“边转边走”更安全，也更容易

控制转完后车身与车道线的相对姿态。

积分本身也有风险：陀螺仪的零偏会随时间累积成角度漂移，调头时间越长误差越大。工程用 6 s 的超时上限来兜底，一旦超时即判定调头失败并锁存停车，而不是无限期地转下去。

5.3 四状态机与失效保护

整个行驶过程由一个四状态机管理（图 8）：FORWARD 负责往程循迹，U_TURN 负责调头，RETURN 负责返程循迹，STOPPED 是锁存停车。状态迁移条件都是显式的，没有任何隐式跳转。

FORWARD 迁移到 U_TURN 的条件是收到 /has_obs 为真；U_TURN 迁移到 RETURN 的条件是累计偏航角与 π 的偏差小于 0.06 rad；任意状态迁移到 STOPPED 的条件有三个：IMU 超过 0.5 s 没有数据、调头超过 6 s 未完成、线心估计超过 0.5 s 没有更新。

STOPPED 是“锁存”的：进入后每个控制周期都发布零速，且不响应任何输入，必须人工排查后重启节点。这是有意为之——在实车运行中，停下来只是损失一点时间，带故障乱跑却可能撞坏车、损坏传感器。对一个以“不撞”为底线的任务来说，锁存停车是唯一合理的失败姿态。



图 8　行驶状态机与失效保护

返程还叠加了两条时间策略：返程 12 s 后进入慢速模式，线速度降到 0.12 m/s、比例增益降到 0.0015，因为返程后半段是出发区的窄弯；返程前 10

s 忽略交通标志，理由已在 4.5 节说明。这两条策略都被写成带注释的常量，便于现场根据场地实际情况调整。

6 系统联调与问题分析

6.1 部署流程与自检

工程在 Windows 开发机上完成编写，但只能在 RDK X5 上编译与运行：编译依赖 dnn_node、hbm_img_msgs、hobot_cv、hbmem 等板端软件包，而这些包随板卡镜像提供。因此开发机的职责仅限于代码编辑与静态检查，真正的验证发生在板端。

板端部署顺序是：先启动底盘（相机、IMU、雷达），再启动本工程的 perception.launch.py，需要接管相机时再起 camera_pipeline.launch.py。启动后按固定顺序自检：用 ros2 topic list 确认 /image、/hbmem_img、/scan、/imu 都存在；用 ros2 topic info 确认消息类型与订阅方数量；用 ros2 topic echo /line_track_center_detection 确认输出的两个分量分别落在 0～1920 与 500～948 之间。最后一条自检尤其重要，它能一次性排除“模型输出未反变换”和“坐标系错位”这两类问题。

此外，2.1 节提到的共享内存链路必须先于感知节点启动。若 hobot_shm 未就绪，/hbmem_img 不会出现，感知节点只会静默地等消息，日志里没有任何报错，很容易被误判为“模型没加载成功”。

6.2 PID 调参的方法论

控制参数列在表 5 中。这一节要说明的不是这些数“是多少”，而是它们“怎么来的”——整定过程遵循手动整定的一般方法，再结合本工程“测量噪声大、没有系统模型”的实际情况做了取舍。

表 5　循迹与调头控制参数

参数	取值	作用
base_speed / min_speed	0.2 / 0.05 m/s	巡航速度与最低速度（前者为前馈项）
kp_angular	0.003	循迹比例增益（像素偏差 → 角速度）
max_angular	0.8 rad/s	角速度限幅
speed_reduction_gain	0.03	转角越大、线速度越低（曲率前馈）
max_slew_rate	0.05 m/s 每周期	线速度变化率限幅
max_point_age	0.5 s	线心估计超时判据
turn_kp / turn_kd	2.0 / 0.5	调头偏航 PD 增益
turn_target / tolerance	π / 0.06 rad	目标转角与到位容差
turn_max_ang / duration	0.6 rad/s / 6.0 s	调头角速度上限与超时

6.2.1 手动整定的三步法

手动整定的经典步骤只有三条：第一步，把 Kp、Ki、Kd 全部置零；第二步，增大 Kp，直到输出在设定值附近开始振荡；第三步，增大 Kd，直到响应刚要出现抖动为止。只有当控制器稳定地停在设定值之上或之下、也就是存在不可接受的稳态误差时，才引入 Ki。

这三步背后是一条清晰的因果链。P 决定“离目标还差多少就使多大劲”，是唯一必须存在的项；D 是对误差变化率的预判，用来压掉 P 引起的超调与振荡；I 负责消除残差，代价是相位滞后与积分饱和。需要警惕的是：Ki 过大会引起 integral windup——设定值大幅变化后，误差积分会累积到超过执行器的最大输出，系统持续过冲并继续增大，直到积攒的误差被“放完”才回落。

还有一条很有用的判据：位置型 PID 需要 Kd，速度型 PID 通常不需要。第二步“加到刚要振荡”本质上是找临界增益，与 Ziegler–Nichols 的临界比例度法同源；实践中不必算出整定表，保留“肉眼看到振荡”这个判据即可——这一点对我们很关键，因为场地条件不允许做阶跃响应实验。

6.2.2 本工程的取舍：P + 前馈，不要 I 和 D

对应到本工程：被控量是“线心像素偏差 → 角速度”，属于位置环，按经典结论应当配 Kd。但 kd_angular 被显式置零，代码注释写的是“KD：抑制震荡，效果一般，放弃”。原因在测量噪声：线心坐标每帧有 3～5 像素的抖动，除以 33 ms 的控制周期后，微分项把噪声放大约 30 倍，D 项输出的抖动幅度与它本该抑制的振荡处于同一量级。换句话说，我们的噪声底噪已经超过了 Kd 能提供的阻尼，按“加到刚要抖动为止”的判据去加，几乎一步都加不动。

I 项同样没有采用。场地是一条连续曲线，控制器不存在“停在某一点”的稳态需求，静差会被下一帧的线心更新自然抹掉；而积分在长弯里会持续累积，出弯时恰恰是最需要反向修正的时刻，windup 造成的过冲最危险。经验上，能用前馈解决的问题不必用积分，本工程的做法是同一思路：用 base_speed 作为线速度前馈，开环给出“这段路该跑多快”，P 只负责纠偏，因此 P 不需要很大。

也正因为放弃了 I 和 D，才需要另外两个机制来补：一是速度-转角耦合系数 speed_reduction_gain，角速度越大说明弯越急，就提前把线速度降下来，相当于一个“曲率前馈”，让 P 在低速、小偏差的工况下工作；二是变化率限幅 max_slew_rate，防止线速度阶跃带来车身俯仰。

6.2.3 执行器饱和与限幅

执行器饱和是一个容易被忽略的环节：当控制输出撞到上限，反馈控制器等效于增益被临时减小，系统响应由指数逼近退化成线性逼近——直观表现就是“方向打死了，车却不转”。

工程里的两处限幅正对应这件事。max_angular = 0.8 rad/s 是角速度上限：一旦长期贴在这个值上，说明 kp 与实际需要的转角不匹配，此时继续加 kp 只会让控制更早饱和，而不是更快入弯。max_slew_rate = 0.05 m/s 每周期是线速度的变化率上限，等效加速度约 1.5 m/s^2，它把“电机给不出更大加速

度”这个物理约束显式写进了控制器，避免指令与执行长期不一致。

由此得到一条实用判据：正常弯道不应打满角速度上限；如果打满，先降 base_speed，再谈 kp。限幅在这里不是保护电机的手段，而是把“物理上给不出更大指令”这件事摊开写进代码。

6.2.4 整定顺序、判据与结果

按上述方法，实际整定顺序是“先速度、再增益、最后限幅”，每一步都以肉眼可判的现象作为判据，见表 6。

表 6　整定顺序、观察量与判据

顺序	调整对象	观察什么	判据	结果
1	base_speed（前馈）	直线段是否匀速、车头是否点头	能稳定看清线、不明显俯仰	0.2 m/s
2	kp_angular	直道是否左右摆、弯道是否跟得上	直道不摆、弯道不冲出	0.003
3	kd_angular	线心抖动是否被放大	不出现高频抖动（否则回到 0）	0（弃用）
4	speed_reduction_gain	入弯是否减速、弯中是否丢线	弯中不丢线	0.03
5	max_angular / max_slew_rate	正常弯道是否打满	不饱和	0.8 rad/s / 0.05
6	Ki	是否长期停在设定值一侧	无稳态需求，不引入	0

具体过程是：先把 kp 置零，只给 base_speed，从 0.05 m/s 起逐步加到 0.2 m/s，确认直线段不点头、线心稳定在画面中部；再恢复 kp，从 0.001 起每次加 0.0005，直到直道上出现可辨认的左右摆，然后退回一档，落到 0.003。按第二步的判据，这个值比“刚要振荡”略小，属于略欠阻尼但稳定的一侧。

第三步尝试加 kd：从 0.01 起，D 项一介入线心抖动立刻被放大，车出现高频抖动，按前面的判据这已经越过“刚要抖动”的边界，于是直接回到 0。这也反过来验证了 6.2.2 节的判断——不是方法不对，是测量噪声不允许。

kp 取 0.003 有一个可以核算的物理含义：100 像素的偏差对应 0.3 rad/s 的角速度，在 0.2 m/s 下车身转弯半径约 0.67 m，与场地最小弯半径相当。再大就会在直道上左右摆，再小则入弯不及。这个“用转弯半径反推增益量级”的算法，比纯试凑更容易解释，也便于换车后重新定标。

6.2.5 方法论的边界

方法本身也有边界：PID 的启发式整定适合“对系统动态一无所知”的场合；一旦有模型，更好的做法是先辨识出增益与时间常数，再用前馈加反馈；而且 PID 只适用于单输入单输出。本工程恰好落在“没有模型”这一侧：小车的转向增益随地面摩擦、胎压、电量变化，今天辨识出的模型明天就未必准，因此选择保守的经验值而不是精确的模型，是权衡后的结果，不是偷懒。

如果要再往前走一步，可行路径是用同一段场地录下“偏差—角速度—实际轨迹”的三元组，拟合一个一阶模型（增益 K、时间常数 τ），用 K 直接算出前馈项、用 τ 定出 kp 的上界。这属于第 7.2 节所说的“把经验值变成可解释的值”。

6.3 典型问题与处理

联调中遇到的典型问题整理在表 7 中。它们的共同点是：现象出现在控制层，根因却往往在数据、接口或工具链里。

表 7　联调中的典型问题、原因与处理

现象	可能原因	处理措施
线心坐标恒在图像最左侧，车持续打死方向	模型输出未反变换回像素	按 (x+1)×960 与 500+(y+1)×224 映射，并加有效性检查
直道上左右频繁摆动	比例增益过大或速度突变	降低 kp_angular，加入变化率限幅
弯道中线心丢失	车头俯仰导致 ROI 内无线	转弯降速，必要时缩小 ROI 高度
标志一闪而过未响应	单帧判定 + 控制时延	连续 2 帧防抖 + 控制端保持 1 s

现象	可能原因	处理措施
调头停在半路	角速度为 0 时静摩擦卡住	设 0.2 rad/s 的角速度下限
返程起步被标志钉住	出发区标志牌或地面反光	返程前 10 s 忽略交通标志
训练脚本报 CUDA 不可用	PyTorch 装成 CPU 版，或显卡架构 sm_120 不被旧轮子支持	换与 CUDA 12.8 匹配的轮子；退路是退回 CPU 训练
量化工具链拒绝导入 ONNX	IR/opset 过新，含 noop_with_empty_axes、allowzero 等新属性	降 opset 到 11、IR 到 9，删新属性并补 shape 输入

把感知、训练与部署放在同一个工程里调试，最大的价值也在这里——可以顺着一条话题、一条转换日志，从发布端一直追到训练脚本，而不是在三套互不相干的工具里各查一遍。

6.4 与课题目标的对应关系

把设计映射回表 1 的四项任务：车道循迹由 ResNet18 回归加 PD 控制承担，正向与逆向两圈依靠“往程—调头—返程”的状态机在一次上电内连续完成；交通标志响应由 HSV 分割、帧间防抖与速度覆盖共同保证，其中停车标志只在返程生效的约束，对应“结束前识别到停车标志并停车”这一要求；障碍识别与调头由雷达扇区聚类与 IMU 闭环完成；运行效率则取决于巡航速度、状态切换的干脆程度以及标志响应的果断程度。

需要如实说明的是：本报告完成的是算法与工程实现，实车表现还取决于场地光照、地面反光、电池电量等现场条件，必须以现场测试为准。工程在 Windows 开发机上只完成了 Python 语法、包元数据与文件清单的静态检查，colcon build、模型加载与实车行驶均未在开发机上验证过。这一点在把工程交给下一届同学时同样需要写明，避免把“代码能跑”误当成“车能跑”。

7 结论与展望

7.1 结论

本课题以一台差速底盘小车为载体，把课堂上的图像处理、目标检测、ROS 2 通信与控制理论串成了一条可运行的链路，形成了三个可以直接复用的成果。

第一，建立了一套“低成本标注 + 端到端回归”的车道线感知方案。752 张图、每张一个点、文件名即标签，配合 ResNet18 的双输出回归，在保证实时性的同时把标注成本压到了最低；数据分布、归一化常数与 ROI 三者的绑定关系被显式写进代码与文档，避免了下游改动引发系统性偏移。

第二，形成了一套按“代价不对称”设计阈值的工程判断。黄色标志面积阈值低到 10 像素²、红蓝标志取 600 像素²，看似不合理，实则是按“漏检丢掉一次动作、误检只损失几秒”的代价不对称反推出来的；同样，避障节点在传感器异常时默认报“有障碍”，也是同一种思路。

第三，把安全性做进了状态机而不是事后补丁。四状态机、三条超时判据、锁存停车与零速兜底，构成了一个“出错就停”的底线；对一个要求连续跑完两圈、中途不能脱线的任务来说，这个底线比任何单点优化都更重要。

7.2 不足与展望

当前的不足主要有三方面。其一，车道线模型只回归单点，缺少对“线是否真的存在”的判断，遇到无线路段（如斑马线、路口）时仍会输出一个看似合理的坐标，控制层无法区分“真线”与“幻觉”；可行的改进是增加一个存在性分支，或用多帧一致性检验来否决孤立输出。

其二，标志识别完全依赖颜色阈值，对光照变化的鲁棒性有限，且无法区分颜色相近的不同标志。受 4.2 节所述的时延约束，短期内更适合继续改进颜色分割——例如引入自适应阈值与多帧投票；若将来换用算力更宽裕的平台，

再考虑把检测模型接进来，用同一套话题语义平滑替换。

其三，参数整定仍以人工试凑为主，缺乏系统辨识。后续可以采集“偏差—角速度—实际轨迹”的数据，用最小二乘拟合出车辆的一阶响应模型，再据此整定 PD 参数，把“经验值”变成“可解释的值”。

从更长远的角度看，本课题最大的收获并不是某一个模型或某一段代码，而是理解了完整系统里各环节的耦合方式：标注差 5 个像素会在直道上表现为贴线，归一化常数改一位会让车直接冲出场地，共享内存没起来会让模型“静默失败”。这些经验无法从单科课程里获得，只能在把车真正跑起来的过程中一点点积累。

8 参考文献

[1] 上海交通大学人工智能实践教学中心. AI 智能驾驶课程讲义：图像处理与目标检测[Z]. 2026.

[2] Originbot 操作手册[EB/OL]. https://www.yuque.com/xuelaoshi-c0byt/pulbcg.

[3] FrozenFisher. 2026-Originbot03-CoreRelease[EB/OL]. https://github.com/FrozenFisher/2026-Originbot03-CoreRelease.

[4] HE K, ZHANG X, REN S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 770-778.

[5] MACENSKI S, FOOTE T, GERKEY B, et al. Robot Operating System 2: Design, architecture, and uses in the wild[J]. Science Robotics, 2022, 7(66): eabm6074.

[6] 地平线机器人. RDK X5 与 dnn_node 应用开发文档[EB/OL]. https://developer.d-robotics.cc.